

PRACTICAL - 2

AIM :Implementation and Time analysis of linear and binary search algorithm

binary search algorithm :

binary search
input : sorted array A with n elements key
output : position of key if found, otherwise "element not found"
step 1 : start
step 2 : Read n, sorted array A, and key
step 3 : set low = 0 and high = n-1
step 4 : Find mid $(low + high) / 2$
step 5 : if $A[mid] = key$, display the position
step 6 : if $A[mid] < key$, set low = mid + 1
step 7 : if $A[mid] > key$, set high = mid - 1
step 8 : Repeat steps 4-7 while low $\leq$ high
step 9 : if the element is not found, display, "element not found"
step 10 : stop

```
code :  
#include <iostream>  
using namespace std;  
  
int main()  
{  
    int n, arr[100], key;  
    int low, high, mid;  
  
    cout << "Enter size of array: ";  
    cin >> n;
```

```
cout << "Enter sorted array elements: ";
for(int i = 0; i < n; i++)
    cin >> arr[i];

cout << "Enter element to search: ";
cin >> key;

low = 0;
high = n - 1;

while(low <= high)
{
    mid = (low + high) / 2;

    if(arr[mid] == key)
    {
        cout << "Element found at position " << mid + 1;
        return 0;
    }
    else if(arr[mid] < key)
    {
        low = mid + 1;
    }
    else
    {
        high = mid - 1;
    }
}

cout << "Element not found";

return 0;
}
```

Output

```
Enter size of array: 5
Enter sorted array elements: 2
5
7
35
89
Enter element to search: 5
Element found at position 2
```

time complexity : Step Count / Frequency Count Method)
best case :

Binary Search (Best case)

$$\text{mid} = (\text{low} + \text{high}) / 2$$

if $\text{arr}[\text{mid}] = \text{key}$
return mid

$$T(n) = 1$$
$$= O(1)$$

average case : Key is found after several divisions

```

Binary search
low = 0; -1
high = n-1; -1
while (low <= high) { O(log n)
    mid = (low + high) / 2; -1
    if (arr[mid] == key) -1
    { return mid;
    } else if (arr[mid] < key) -1
    { low = mid + 1; } -1
    else { high = mid - 1; } -1
} return -1; -1
T(n) = O(log n)
    
```

worst case : Key is found at the last division / not found

```

low = 0; -1
high = n-1; -1
while (low <= high) { O(log n)
    mid = (low + high) / 2; -1
    if (arr[mid] == key) -1
    { return mid; -1
    } else if (arr[mid] < key) -1
    { low = mid + 1; } -1
    else { high = mid - 1; } -1
} return -1; -1
T(n) = O(log n)
    
```

linear search algorithm :

Linear Search
Input :- Array A with n elements and a search key
Output :- position of key, otherwise "element not found"
Step 1 :- start
Step 2 :- Read n, array A, and key
Step 3 :- set $i = 0$
Step 4 :- compare $A[i]$ with key.
Step 5 :- if $A[i] = \text{key}$, display the position and go to step 8
Step 6 :- increment i by 1 and repeat steps 4-5 until
Step 7 :- display "element not found"
Step 8 :- stop

code :
#include <iostream>

```
using namespace std;

int main()
{
    int n, arr[100], key;

    cout << "Enter size of array: ";
    cin >> n;

    cout << "Enter array elements: ";
    for(int i = 0; i < n; i++)
        cin >> arr[i];

    cout << "Enter element to search: ";
    cin >> key;

    for(int i = 0; i < n; i++)
    {
        if(arr[i] == key)
        {
            cout << "Element found at position " << i + 1;
```

```

        return 0;
    }
}

cout << "Element not found";

return 0;
}
    
```

Output

```

Enter size of array: 5
Enter array elements: 34
67
23
65
1
Enter element to search: 24
Element not found
    
```

time complexity (Step Count / Frequency Count Method)

best case :

element is found at the first position
 for (i=0; i<n; i++) {
 if (arr[i] == key) {
 return i; }
 }
 T(n) = 1
 = O(1)

average case : Element is somewhere in the middle

```
linear search
for (i=0; i<n; i++) - n
{
    if (arr[i] == key) - 1
    {
        return i; - 1
    }
}
return -1; - 1
T(n) = O(n)
```

worst case : Element is at the last position / not found

```
linear search
for (i=0; i<n; i++) - n
{
    if (arr[i] == key) - 1
    {
        return i; - 1
    }
}
return -1; - 1
T(n) = O(n)
```